S.B.V.P.SAMAJ'S

SAHAKAR MAHARSHI BHAUSAHEB SANTUJI THORAT ARTS, COMMERCE & SCIENCE COLLEGE SANGAMNER DIST- AHILYANAGAR

Academic Year 2026-2027

An

PROJECT REPORT

ON

" Farmer to Customer Marketplace with AI"

in partial fulfillment of the requirement of the

T.Y.B.Sc. Computer Science Semester V

Under The Guidance of

Prof. Pratiksha Walke

Submitted By

" Adangale Tejas Janku"

S.B.V.P.SAMAJ'S

SAHAKAR MAHARSHI BHAUSAHEB SANTUJI THORAT ARTS, COMMERCE & SCIENCE COLLEGE SANGAMNER DIST- AHILYANAGAR

Department of Computer Science Project

Report

Academic Year 2026-2027

CERTIFICATE

This certificate is awarded to Mr. Adangale Tejas Janku in appreciation of their meritorious performance in the project titled on "Farmer to Customer Marketplace with AI" This project was completed as part of T.Y.B.Sc. Computer Science Semester V practical requirements of Savitribai Phule Pune University during the Academic Year 2026-2027.

Prof. Pratiksha Walke — **Mr. Thakare M.V**

(Project Guide) — **(HOD)**

Internal Examiner — **External Examiner**

ACKNOWLEDGEMENT

We deem it a proud privilege and a profound pleasure to express our sincere gratitude to our project guide, **Prof. Walke Mam**, under whose expert guidance we successfully carried out this project work, *"KisanDirect - Farmer to Customer Marketplace with AI"*. Her incisive guidance, practical advice, and continuous encouragement provided us with the confidence to develop this platform and smoothly integrate the "Krushi-Mitra" AI chatbot.

We shall remain highly grateful to her for providing a strong academic atmosphere. Her focus on discipline and dedication kept us highly motivated and concentrated on our project work from start to finish.

We would also like to thank all our respected faculty members for their inspirational help and valuable suggestions, which helped shape this software and the final report into its present professional form.

Finally, our deepest sense of gratitude and heartfelt thanks go to our parents and family members who constantly supported, blessed, and motivated us throughout this entire journey.

Name of the Student:

Mr.Adangale Tejas Janku.

ABSTRACT

The agricultural sector plays a vital role in rural employment and economic development. However, the traditional agricultural supply chain relies heavily on multiple middlemen. This long chain results in severe inefficiencies, leading to low profit margins for farmers and highly inflated prices for end consumers.

To solve this problem, **"KisanDirect - Farmer to Customer Marketplace with AI"** is developed as a dynamic, web-based Agri-Tech platform. This project provides a direct digital bridge between local farmers and buyers. The system enables farmers to easily register, list their fresh produce, manage inventory, and set dual-pricing options (Retail price per Kg and Wholesale price for bulk buyers). Simultaneously, it allows customers to browse the catalog, manage their shopping carts, and place orders efficiently without any third-party interference.

A major highlight of this project is the integration of an intelligent AI chatbot, **"Krushi-Mitra AI"**. This smart assistant provides all users with 24/7 real-time support, instantly answering queries related to agricultural products, farming guidance, and platform navigation.

The main objective of this project is to eliminate the exploitation caused by middlemen, ensure transparency, and improve overall efficiency in agricultural trade. By implementing a structured, secure, and user-friendly web application, KisanDirect empowers farmers financially and provides consumers with fresh produce at fair prices.

Overall, the system aims to support sustainable agricultural practices and drive rural economic growth through digital transformation and Artificial Intelligence.

INDEX			
Sr. No.	**Title**	**Page No.**	**Signature of the Instructor**
1	**Preliminary Investigation** 1.1 Problem Identification 1.2 Problem Statement / Definition 1.3 Purpose/objective and goals 1.4 Feasibility study 1.5 Project Scope and Limitations		
2	**Requirement Specification** 2.1 System Requirement - Software/hardware specifications - Platform, RAM, Memory storage (if required - specification of server / device) 2.2 Technical Requirement - Programming language, Tools if any 2.3 Functional Requirements 2.4 Data Requirements (As per the project requirement, student can include - Performance requirement & security requirements etc.)		
3	**Database Design** 3.1 Identify end users of the system 3.2 Identify the entities and attributes through ER diagrams 3.3 Identify all tables, fields, relationships between table		
4	**System Design** 4.1 Class diagram 4.2 Object diagram 4.3 Component diagram 4.4 Deployment diagram 4.5 Use case diagram 4.6 Activity diagram 4.7 Sequence diagram		
5	**Input and Output screen Layout**		
6	**Bibliography / References**		

INTRODUCTION

Agriculture is the backbone of the Indian economy, playing a vital role in providing food, employment, and raw materials. However, in many rural areas, farmers face severe challenges such as a lack of direct market access, heavy dependence on middlemen, and unfair price fluctuations. The traditional agricultural supply chain relies on a long network of brokers, which drastically reduces the farmers' profit margins and increases the cost for end consumers.

To overcome these challenges, **"KisanDirect - Farmer to Customer Marketplace with AI"** has been designed as an advanced, web-based e-commerce application. It provides a direct digital platform where farmers can easily register, showcase their fresh produce, and set their own fair prices using a dual-pricing model (Retail price per Kg and Wholesale price for bulk buyers). Simultaneously, customers and bulk buyers can browse these agricultural products, manage their shopping carts, and place orders directly from the farmers.

A unique and highly advanced feature of this platform is the integration of **"Krushi-Mitra AI"**, an intelligent chatbot. Instead of relying on manual customer care, users can interact with this AI assistant 24/7. It provides farmers with instant agricultural guidance and assists customers with real-time product inquiries, making platform communication smooth and effortless.

By offering a centralized, transparent, and organized digital marketplace, KisanDirect completely eliminates intermediaries, ensuring high price realization for farmers and fresh quality for buyers. This project aims to revolutionize agricultural marketing by introducing a user-friendly, AI-powered digital solution that supports financial empowerment, sustainable growth, and the modern digital transformation of the agricultural sector.

1.Preliminary Investigation

❖ Overview:

The preliminary investigation is the initial phase of the system development life cycle. In this phase, we analyze the current traditional agricultural supply chain to understand its flaws, challenges, and inefficiencies. By studying how farmers currently sell their crops and how consumers purchase them, we identified a massive gap between the two parties. This investigation lays the foundation for "KisanDirect", proposing a digital Agri-Tech marketplace to bridge this gap, eliminate exploitation, and create a transparent, AI-assisted, digital-first ecosystem for agricultural trade.

1.1 Problem Identification:

In the current agricultural system, the journey of fresh produce from the farm to the consumer involves multiple intermediaries (middlemen, commission agents, wholesalers, and retailers). This traditional supply chain creates several significant problems:

- **Low Profit Margins for Farmers:** Due to the high commission rates taken by multiple middlemen, farmers receive a very small fraction of the final selling price.
- **High Costs for Consumers:** By the time vegetables and fruits reach the end customer or hotels, the prices are highly inflated.
- **Loss of Freshness:** The long chain of intermediaries delays the delivery process, causing the fresh produce to lose its quality and freshness.
- **Lack of Market Access:** Local farmers lack a dedicated digital platform to showcase their produce directly to bulk buyers and retail customers.
- **Lack of Real-Time Guidance & Support:** Farmers often lack immediate access to agricultural advice or platform support, while consumers struggle to get instant answers about product quality or sourcing.

1.2 Problem Statement / Definition:

"To design and develop a web-based Agri-Tech marketplace named 'KisanDirect' that eliminates the chain of middlemen by providing a direct digital platform for farmers to sell their fresh produce to retail consumers and bulk buyers (like hotels). The system is enhanced with an Artificial Intelligence (AI) chatbot to provide 24/7 smart assistance,

ensuring fair pricing, higher profit margins for farmers, fresh quality for consumers, and a highly interactive, user-friendly e-commerce experience."

1.3 Purpose, Objectives, and Goals:

The primary purpose of KisanDirect is to empower farmers digitally and provide a transparent shopping experience for customers.

- **Goals & Objectives:**

- **Direct Connectivity:** To connect farmers directly with end consumers and wholesale buyers.
- **Dual Pricing Model:** To provide dynamic pricing options (Retail price per Kg and Wholesale price per Box/Bulk) to cater to different types of customers.
- **User-Friendly E-commerce Interface:** To deliver a professional, "Flipkart-like" shopping experience with smooth cart management, product filtering, and detailed invoice generation.
- **AI Assistance:** To integrate an AI-powered chatbot ("Krushi-Mitra AI") to assist users with their queries in real-time, offering instant agricultural tips to farmers and product information to customers.
- **Empowerment:** To help farmers build their own customer base and increase their income by saving on middleman commissions.

1.4 Feasibility Study:

Before developing the system, a feasibility study was conducted to ensure the project is practical and achievable.

- **Technical Feasibility:** The project is technically feasible as it uses widely accepted and accessible technologies. The backend is powered by Python (Flask framework), which handles dynamic routing and database management easily. The frontend utilizes standard HTML, CSS (Glassmorphism design), and JavaScript. Furthermore, the integration of the "Krushi-Mitra AI" chatbot is technically viable using external RESTful APIs, which seamlessly connect with the Flask backend.
- **Operational Feasibility:** The platform is designed with a highly intuitive, clean user interface. Features like categorized searches, easy cart management, and one-click invoice generation make it very easy for both farmers and consumers to operate the system. The presence of the AI chatbot further enhances operational feasibility by acting as a 24/7 virtual guide for users who might need help navigating the platform.

- **Economic Feasibility:** The project is economically feasible because it utilizes open-source technologies (Python, Flask, SQLite) and free-tier API services for the AI. There are no high licensing costs involved. For the users, it increases revenue for farmers and saves money for consumers, making it a highly valuable platform.

1.5 Project Scope and Limitations Scope:

- The system allows role-based registration and secure login for Farmers, Customers, and Admins.
- Farmers can list their produce, update stock, and set retail/wholesale prices.
- Customers can search for products by category, add items to a dynamic cart, and generate a professional, printable billing invoice.
- The platform includes interactive features like the "Krushi-Mitra" AI chatbot for real-time query resolution and a responsive, glassmorphism-based UI for seamless navigation.

➢ **Limitations:**

- **Logistics and Delivery:** The current system acts as a marketplace to connect buyers and sellers. The physical delivery of the produce is to be handled externally by the farmer or the buyer.
- **Payment Gateway:** The system currently simulates the order placement process (Cash on Delivery or basic checkout) and does not yet have a fully integrated live bank payment gateway.
- **AI Dependency & Internet Requirement:** The platform requires an active internet connection to access the marketplace. Additionally, the intelligence of the chatbot relies on an external AI provider; hence, responses are generated based on available API data and require network connectivity to function.

2.Requirement Specification

❖ Overview:

This chapter outlines the specific hardware, software, technical, and functional needs required to develop and run the "KisanDirect" Agri-Tech marketplace efficiently. It ensures that the system is built on a stable architecture and meets all the operational, security, and performance expectations of its end-users, including the seamless integration of advanced Artificial Intelligence features.

2.1 System Requirement (Software/Hardware Specifications):

To ensure smooth development and deployment of the application, the following system specifications are recommended for the server and client machines.

➢ **Hardware Specifications (Minimum / Recommended):**

- **Processor:** Intel Core i3 / AMD Ryzen 3 (Core i5 or equivalent recommended for faster processing).
- **RAM (Memory):** Minimum 4 GB (8 GB or higher recommended for multitasking and running the local server smoothly).
- **Storage (Hard Drive/SSD):** Minimum 500 MB of free disk space for project files, virtual environments, and database storage.
- **Network:** An active and stable high-speed Internet connection is strictly required for fetching real-time responses via the External AI API (e.g., Gemini/OpenAI) and general web browsing.

➢ **Software Specifications:**

- **Operating System:** Windows 10/11, macOS, or any standard Linux distribution (Ubuntu).
- **Web Browser:** Google Chrome, Mozilla Firefox, Microsoft Edge, or Safari (Needs to support modern CSS like backdrop-filter for Glassmorphism and JavaScript Fetch APIs for the chatbot).

2.2 Technical Requirement (Programming Languages and Tools):

The project utilizes a modern, lightweight, and robust technology stack to deliver a seamless e-commerce experience.

- **Frontend Technologies (Client-Side):**

- **HTML5:** For structuring the web pages and layouts.
- **CSS3:** Used for advanced styling, specifically implementing the modern 'Glassmorphism' (frosted glass) UI theme, responsive grid layouts, and smooth animations.
- **JavaScript (Vanilla JS):** Crucial for client-side interactivity, managing asynchronous requests (AJAX/Fetch) for the AI chatbot widget without reloading the page, handling pop-up modals, and dynamic UI updates.
- **FontAwesome:** For integrating vector icons (e.g., shopping cart, tractor, user profile, robot for AI) across the website.

- **Backend Technologies (Server-Side):**

- **Programming Language:** Python 3.x.
- **Web Framework:** Flask (A lightweight WSGI web application framework used for routing, session management, rendering templates, and serving as a bridge between the frontend chatbot UI and the external AI API).

- **Database:**

- **SQLite:** A C-language library that implements a small, fast, self-contained SQL database engine. Integrated using Flask-SQLAlchemy (ORM).

- **Development Tools (IDE):**

- **Visual Studio Code (VS Code):** The primary Integrated Development Environment used for writing, testing, and debugging the code.

2.3 Functional Requirements:

Functional requirements define the specific behaviors and capabilities that the KisanDirect system must provide to its users.

- **User Authentication System:** The system must allow users to securely register and log in with specific roles (Farmer, Customer, or Admin) and manage their sessions.

- **Product Management (Farmer Role):** Farmers must be able to add fresh produce, define available stock, and set dual-pricing options (Retail Price per Kg and Wholesale Price per Box).
- **Shopping & Navigation (Customer Role):** Customers must be able to browse products, filter them by categories (e.g., Vegetables, Fruits, Grains), and add items to a dynamic shopping cart without losing their current page state.
- **Order & Invoice Generation:** The system must calculate accurate subtotals, apply discounts if any, and generate a print-friendly, professional billing invoice for the user.
- **AI Chatbot Integration ("Krushi-Mitra AI"):** The system must feature a built-in AI assistant. It must functionally capture user input via the chat UI, send the text to the Flask backend, retrieve a generated response from the external AI model, and display the answer to the user in real-time.

2.4 Data, Performance & Security Requirements

➢ **Data Requirements:**

- The system must accurately store and manage user profiles, product catalogs, pricing tiers, and transaction history in a structured relational database (SQLite).
- (Optional) The system should be capable of logging chat histories or specific user queries made to the AI for future reference and system improvement.
- The data must be retrieved quickly to display dynamic product grids on the home page.

➢ **Performance Requirements:**

- The web application should be highly responsive, loading product images and pages within a few seconds.
- The AI Chatbot API response time should be optimized to provide answers within 2-4 seconds to ensure a conversational feel.
- The UI must adapt seamlessly to different screen sizes (desktop, tablet, and mobile) without breaking the layout.

➢ **Security Requirements:**

- **Session Management:** User sessions must be strictly handled so that sensitive pages (like Farmer Dashboard or User Profile) are inaccessible without a valid login.
- **Data Protection:** Implementation of SQLAlchemy ORM automatically helps in preventing standard SQL Injection attacks.

- **Secure Routing & API Keys:** Functions like 'Add to Cart' or 'Checkout' must verify authentication before processing any backend data modifications. Furthermore, the external AI API keys used by the Flask backend must be stored securely (e.g., using environment variables) and should never be exposed to the client-side browser.

3. Database Design

❖ **Overview:**

The database design phase is critical for structuring how information is stored, retrieved, and managed within the "KisanDirect" application. Since the platform handles dynamic product listings, dual-pricing models, role-based user authentication, and AI-driven conversational data, a robust relational database architecture (managed via SQLite and SQLAlchemy ORM) is required to ensure data integrity, security, and fast access.

3.1 Identify End Users of the System

The end users are the primary actors who will interact directly with the KisanDirect web application. The database is designed to store distinct profiles, access rights, and transaction records for each of these user categories. The system primarily identifies the following end users:

1. **Farmers (Sellers / Producers)**
    - **Role Description:** Farmers are the primary suppliers of the marketplace. They use the platform to directly reach consumers without relying on traditional middlemen.
    - **System Interaction:** Farmers require access to a dedicated dashboard where they can add new agricultural products, update available stock quantities, and set dual pricing (Retail price per Kg and Wholesale price per Box). Furthermore, they can interact with the "Krushi-Mitra AI" chatbot for market trends and agricultural queries, generating chat history data.
2. **Retail Customers (End Consumers)**
    - **Role Description:** These are individual buyers or households looking to purchase fresh, high-quality vegetables, fruits, and grains in smaller, daily-use quantities.
    - **System Interaction:** Retail customers interact with the system by browsing the product catalog, using category filters, managing their dynamic shopping cart, placing orders, and generating retail billing invoices. They also frequently interact with the AI Chatbot to ask questions

regarding product freshness or delivery, seamlessly logging chat interactions into the database.

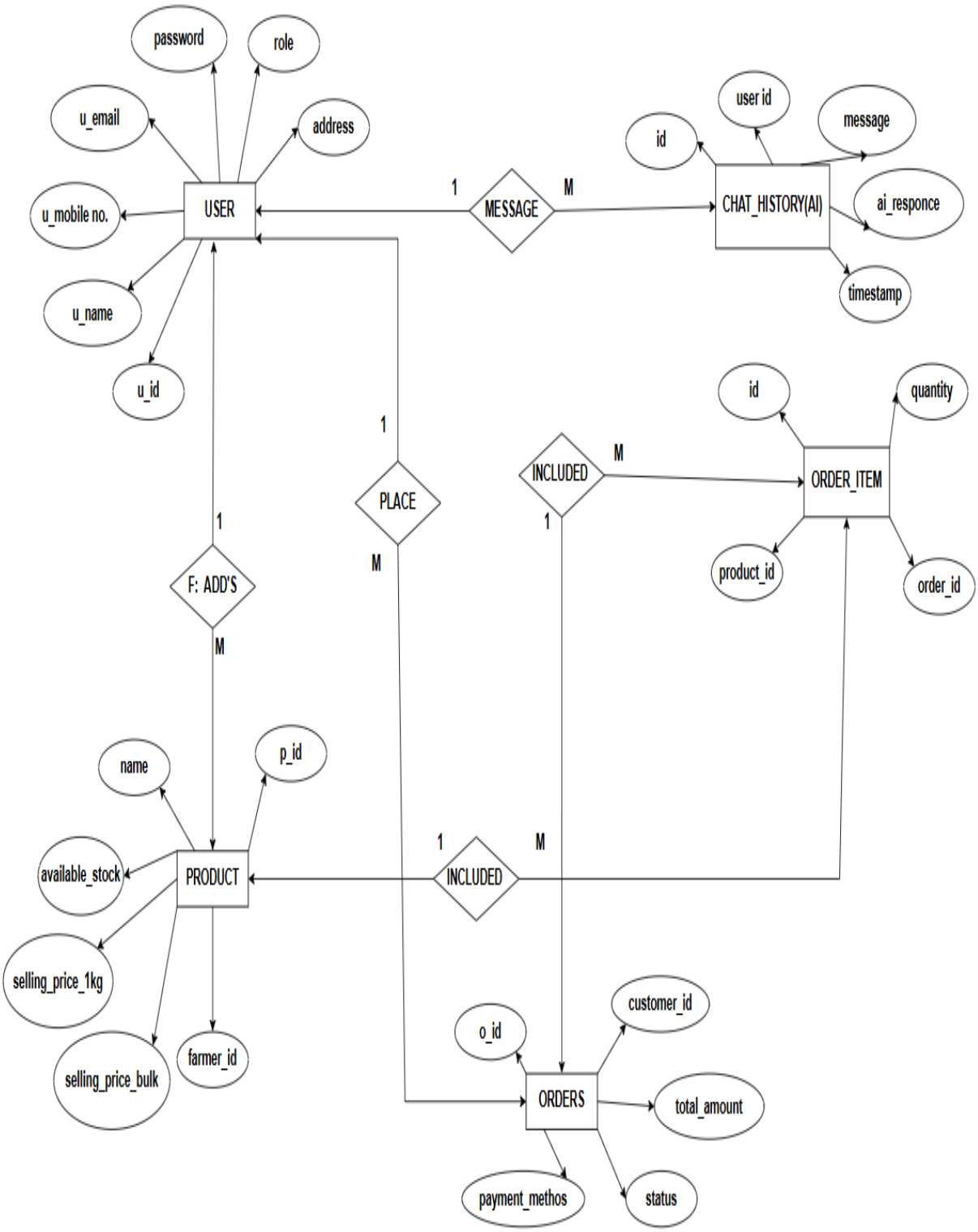
3. **Bulk Buyers (Hotels / Restaurants / Caterers)**
    - **Role Description:** These are Business-to-Business (B2B) entities that require fresh produce in large quantities on a regular basis.
    - **System Interaction:** While they share the same interface as retail customers, their interaction focuses on the 'Wholesale' purchasing options. The database must seamlessly process their transactions using the secondary pricing tier (Bulk/Box pricing) and handle large-volume order tracking.
4. **System Administrator (Admin)**
    - **Role Description:** The administrator is the backend manager responsible for the smooth operation, security, and maintenance of the digital marketplace.
    - **System Interaction:** The Admin oversees user registrations, monitors platform transactions, resolves technical disputes, and manages the central database (including monitoring AI chat logs for malicious inputs) to prevent data corruption or unauthorized access.

3.2 Identify the Entities and Attributes (through ER diagrams)

❖ Overview:

An Entity-Relationship (ER) model is used to define the data structure of the "KisanDirect" platform. As illustrated in the provided ER diagram, entities represent the real-world objects in our system, and attributes define the properties or characteristics of those entities. Based on the system's workflow, we have identified five primary entities: USER, PRODUCT, ORDERS, ORDER_ITEM, and CHAT_HISTORY(AI).

❖ **ER-Diagram:**


password
role
u_email
address
user id
message
id
u_mobile no.
USER
1
MESSAGE
M
CHAT_HISTORY(AI)
ai_responce
u_name
timestamp
u_id
id
quantity
1
INCLUDED
M
ORDER_ITEM
PLACE
1
M
F: ADD'S
product_id
order_id
1
M
name
p_id
available_stock
1
M
PRODUCT
INCLUDED
selling_price_1kg
customer_id
o_id
selling_price_bulk
farmer_id
ORDERS
total_amount
payment_methos
status

1. **Entity: USER**
    - **Description:** This entity stores the profile and login credentials of all individuals interacting with the system.
    - **Attributes:** u_id (Primary Key), u_name, u_mobile no., u_email, password, role, and address.
2. **Entity: PRODUCT**
    - **Description:** This entity represents the agricultural goods listed on the marketplace by the farmers.
    - **Attributes:** p_id (Primary Key), name, available_stock, selling_price_1kg, selling_price_bulk, and farmer_id (Foreign Key referencing USER entity).
3. **Entity: ORDERS**
    - **Description:** This entity records the transactional data generated when a customer places an order on the platform.
    - **Attributes:** o_id (Primary Key), customer_id (Foreign Key referencing USER entity), total_amount, payment_methos (Payment Method), and status.
4. **Entity: ORDER_ITEM**
    - **Description:** This entity acts as a detailed breakdown of the products included within a specific order.
    - **Attributes:** id (Primary Key), product_id (Foreign Key), order_id (Foreign Key), and quantity.
5. **Entity: CHAT_HISTORY(AI)**
    - **Description:** This newly introduced entity logs the conversational interactions between users and the "Krushi-Mitra AI" chatbot to maintain context and history.
    - **Attributes:** id (Primary Key), user id (Foreign Key referencing USER entity), message (User's query), ai_responce (Generated response), and timestamp.

3.3 Identify all tables, fields, relationships between table

❖ Overview:

Based on the Entity-Relationship model, the conceptual data is transformed into a relational database schema. The SQLite database for "KisanDirect" consists of five primary tables to ensure data normalization and efficient data retrieval. Below is the detailed schema outlining the relationships.

1. Table Name: Users

- u_id (Integer) - Primary Key, Auto-increment.
- u_name (String/Varchar) - Not Null.
- u_email (String/Varchar) - Unique, Not Null.
- u_mobile no. (String/Varchar) - Not Null.
- password (String/Varchar) - Not Null (Encrypted).
- role (String/Varchar) - 'Farmer', 'Customer', or 'Admin'.
- address (Text) - User's delivery/farm location data.

2. Table Name: Products

- p_id (Integer) - Primary Key, Auto-increment.
- name (String/Varchar) - Not Null.
- available_stock (Float) - Quantity available.
- selling_price_1kg (Float) - Retail price.
- selling_price_bulk (Float) - Wholesale price.
- farmer_id (Integer) - Foreign Key referencing Users(u_id).

3. Table Name: Orders

- o_id (Integer) - Primary Key, Auto-increment.
- customer_id (Integer) - Foreign Key referencing Users(u_id).
- total_amount (Float) - Final bill amount.
- payment_methos (String/Varchar) - (e.g., COD).
- status (String/Varchar) - (e.g., Order Placed, Processing).

4. Table Name: Order_Items (Junction Table)

- id (Integer) - Primary Key, Auto-increment.

- order_id (Integer) - Foreign Key referencing Orders(o_id).
- product_id (Integer) - Foreign Key referencing Products(p_id).
- quantity (Float) - The amount of product ordered.

5. Table Name: Chat_History (AI Integration)

- id (Integer) - Primary Key, Auto-increment.
- user id (Integer) - Foreign Key referencing Users(u_id).
- message (Text) - The exact text prompt asked by the user.
- ai_responce (Text) - The reply generated by the external AI API.
- timestamp (DateTime) - The exact time the conversation occurred.

Relationships Between Tables (As mapped in the ER Diagram):

- **Users (Farmer) ➔ Products:** Type: One-to-Many (1:M). Established via the **"F: ADD'S"** relationship. One Farmer can list multiple Products.
- **Users (Customer) ➔ Orders:** Type: One-to-Many (1:M). Established via the **"PLACE"** relationship. One Customer can place multiple Orders.
- **Users ➔ Chat_History(AI):** Type: One-to-Many (1:M). Established via the **"MESSAGE"** relationship. One User can have multiple interactions with the AI Chatbot.
- **Orders ➔ Order_Item:** Type: One-to-Many (1:M). Established via the **"INCLUDED"** relationship. One Order can contain multiple Order Items.
- **Products ➔ Order_Item:** Type: One-to-Many (1:M). Established via the **"INCLUDED"** relationship. A single Product can be included in multiple different Order Items across the platform.

Data Dictionary

❖ **Overview:**

The Data Dictionary provides a detailed description of all the tables, fields, data types, and constraints used in the SQLite database (managed via Flask-SQLAlchemy) of the KisanDirect system. It includes the structural breakdown of inventory management, dual-pricing, transaction headers, and the AI conversational data required for the "Krushi-Mitra" Chatbot.

Table 1: Users (Stores Admin, Farmer, and Customer Details)

Field Name	Data Type	Size	Constraint	Description
id	Integer	11	Primary Key, Auto Increment	Unique ID for every user.
full_name	Varchar	100	Not Null	Legal name of the user.
email	Varchar	100	Unique, Not Null	Email address for login.
mobile	Varchar	15	Not Null	Contact number of the user.
password	Varchar	255	Not Null	Encrypted password string.
role	Varchar	20	Not Null	Defines if user is 'Admin', 'Farmer', or 'Customer'.
address	Text	-	Nullable	Detailed delivery or farm address.
city	Varchar	50	Nullable	City of the user.
pincode	Integer	6	Nullable	6-digit postal code.

Table 2: Products (Stores Inventory & Dual-Pricing Details)

Field Name	Data Type	Size	Constraint	Description
id	Integer	11	Primary Key, Auto Increment	Unique ID for every product.
name	Varchar	100	Not Null	Name of the agricultural product.
category	Varchar	50	Not Null	E.g., Vegetables, Fruits, Grains.
available_stock	Float	-	Not Null	Quantity currently available with farmer.
unit_1kg	Varchar	20	Not Null	Retail unit (e.g., 'Kg').
selling_price_1kg	Float	-	Not Null	Retail price per Kg.
unit_bulk	Varchar	20	Not Null	Wholesale unit (e.g., 'Box', 'Quintal').
selling_price_bulk	Float	-	Not Null	Discounted wholesale price.

Field Name	Data Type	Size	Constraint	Description
image_file	Varchar	255	Nullable	File name/URL of the product image.
farmer_id	Integer	11	Foreign Key	References **id** in Users table.

Table 3: Orders (Stores Main Transaction Header)

Field Name	Data Type	Size	Constraint	Description
id	Integer	11	Primary Key, Auto Increment	Unique invoice/tracking number.
customer_id	Integer	11	Foreign Key	References **id** in Users table.
total_amount	Float	-	Not Null	Final calculated bill amount.
payment_method	Varchar	50	Not Null	E.g., Cash on Delivery (COD).
status	Varchar	50	Default 'Processing'	Current status (Placed, Processing, Delivered).
created_at	DateTime	-	Default Current Time	Timestamp when order was placed.

Table 4: Order_Items (Junction Table mapping Orders and Products)

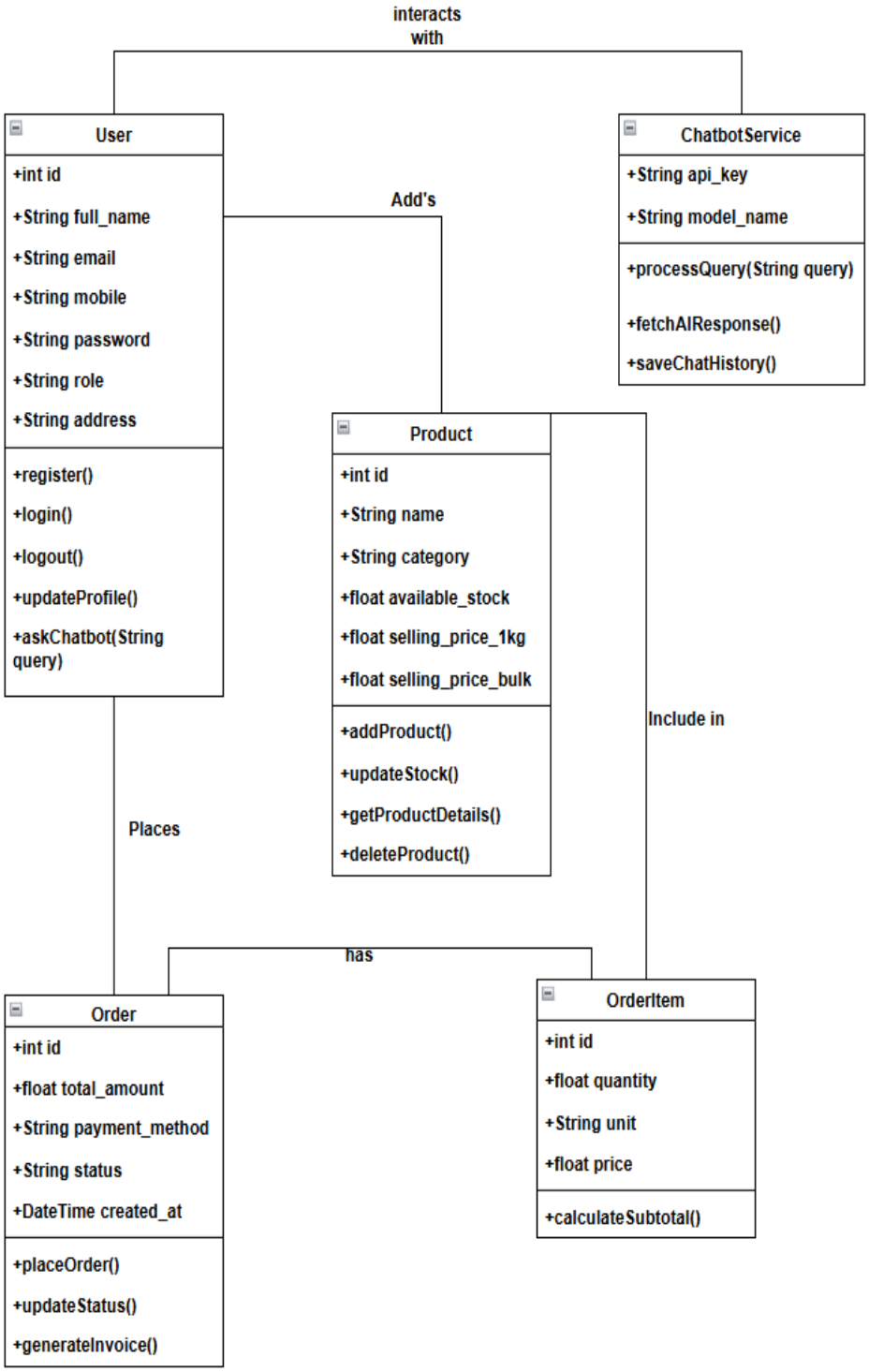
Field Name	Data Type	Size	Constraint	Description
id	Integer	11	Primary Key, Auto Increment	Unique ID for the specific cart item.
order_id	Integer	11	Foreign Key	References **id** in Orders table.
product_id	Integer	11	Foreign Key	References **id** in Products table.
quantity	Float	-	Not Null	Amount of product purchased.
unit	Varchar	20	Not Null	Unit purchased (e.g., 'Kg' or 'Box').
price	Float	-	Not Null	Locked price at the time of order placement.

Table 5: Chat_History (Krushi-Mitra AI Integration)

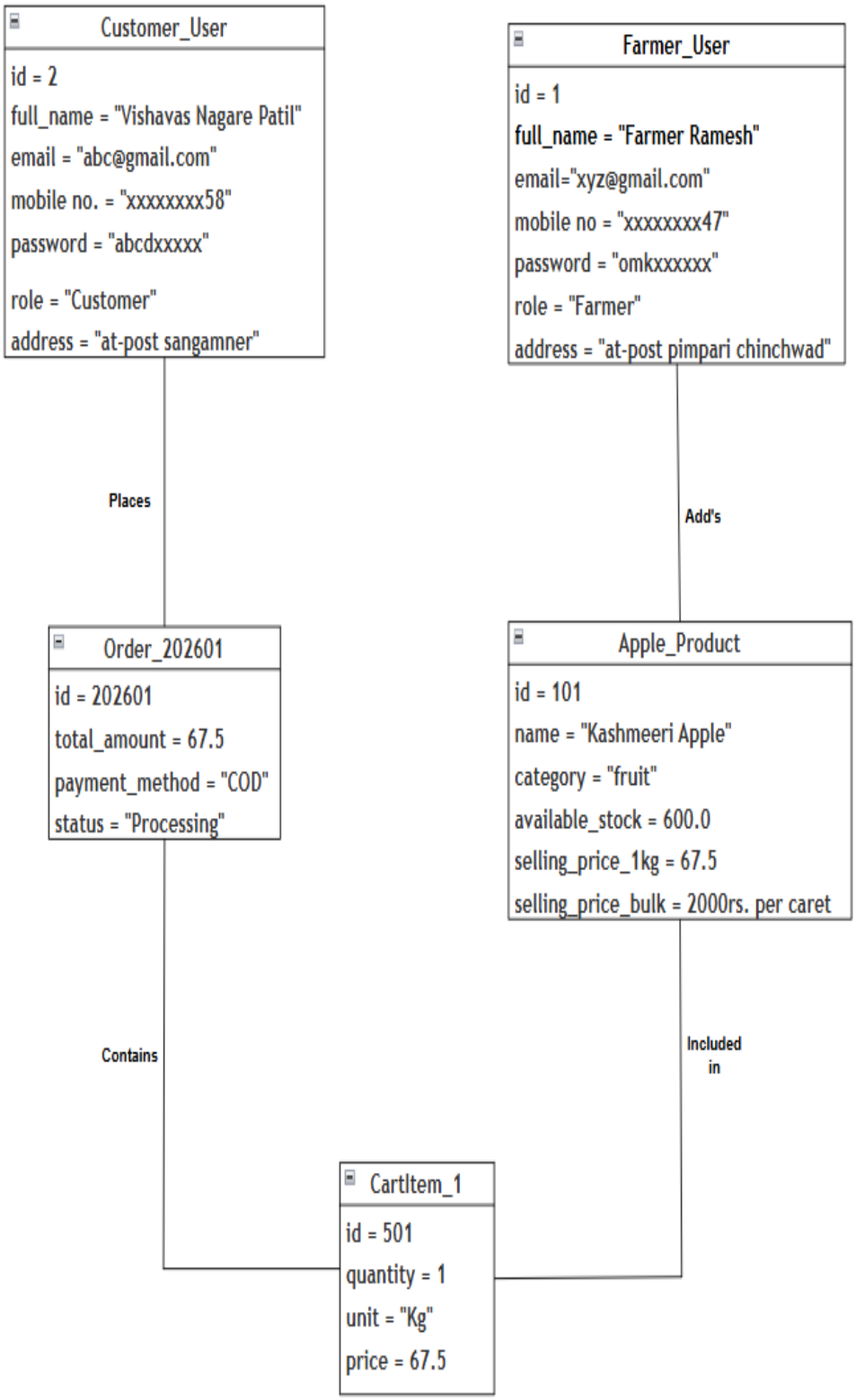
Field Name	Data Type	Size	Constraint	Description
id	Integer	11	Primary Key, Auto Increment	Unique ID for the chat record.
user_id	Integer	11	Foreign Key	References **id** in Users table.
message	Text	-	Not Null	The exact text prompt or query asked by the user.
ai_response	Text	-	Not Null	The smart response generated by the external AI API.
timestamp	DateTime	-	Default Current Time	Timestamp recording when the conversation occurred.

4. System Design

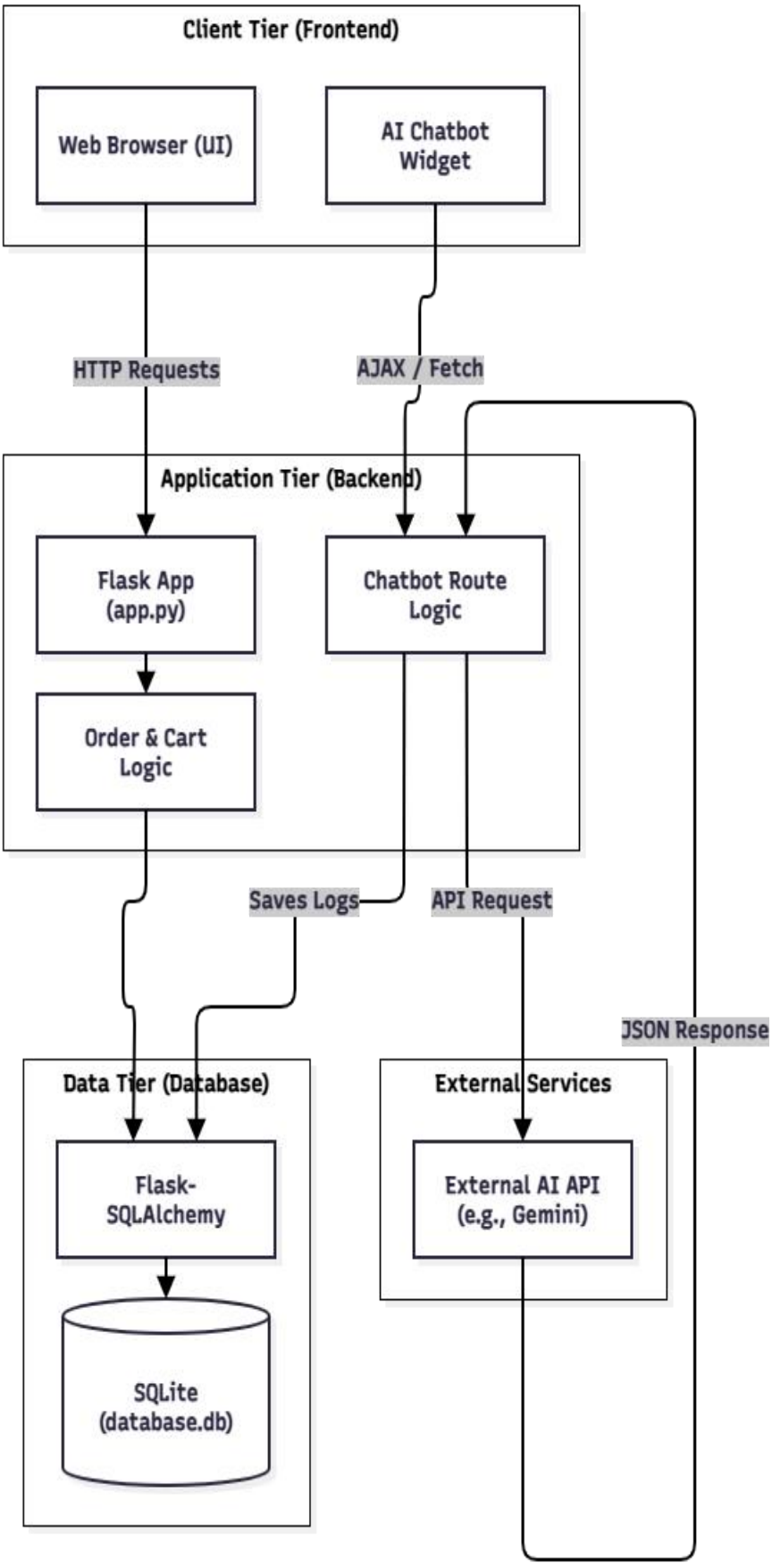
4.1 Class Diagram



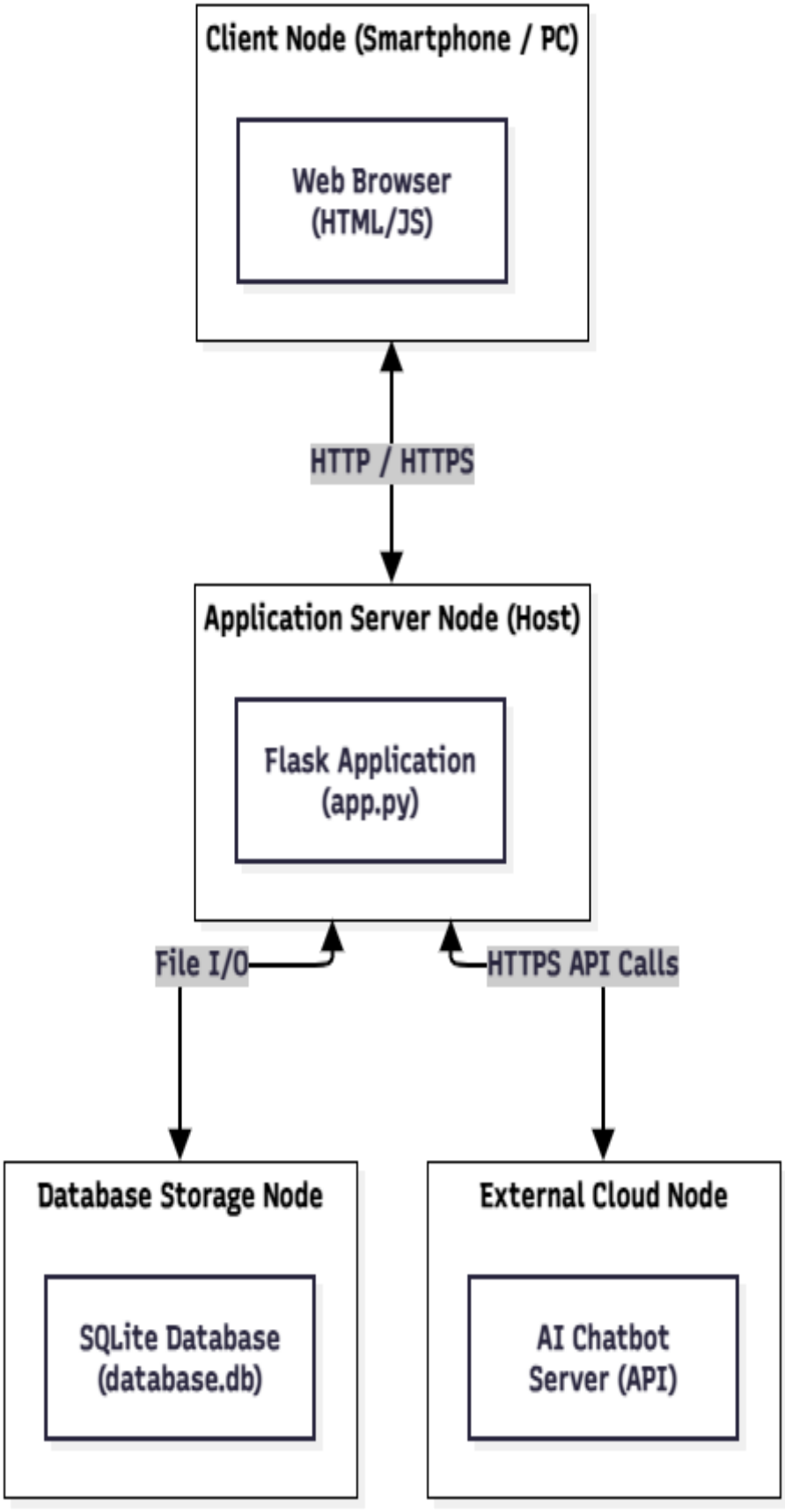
4.2 Object Diagram



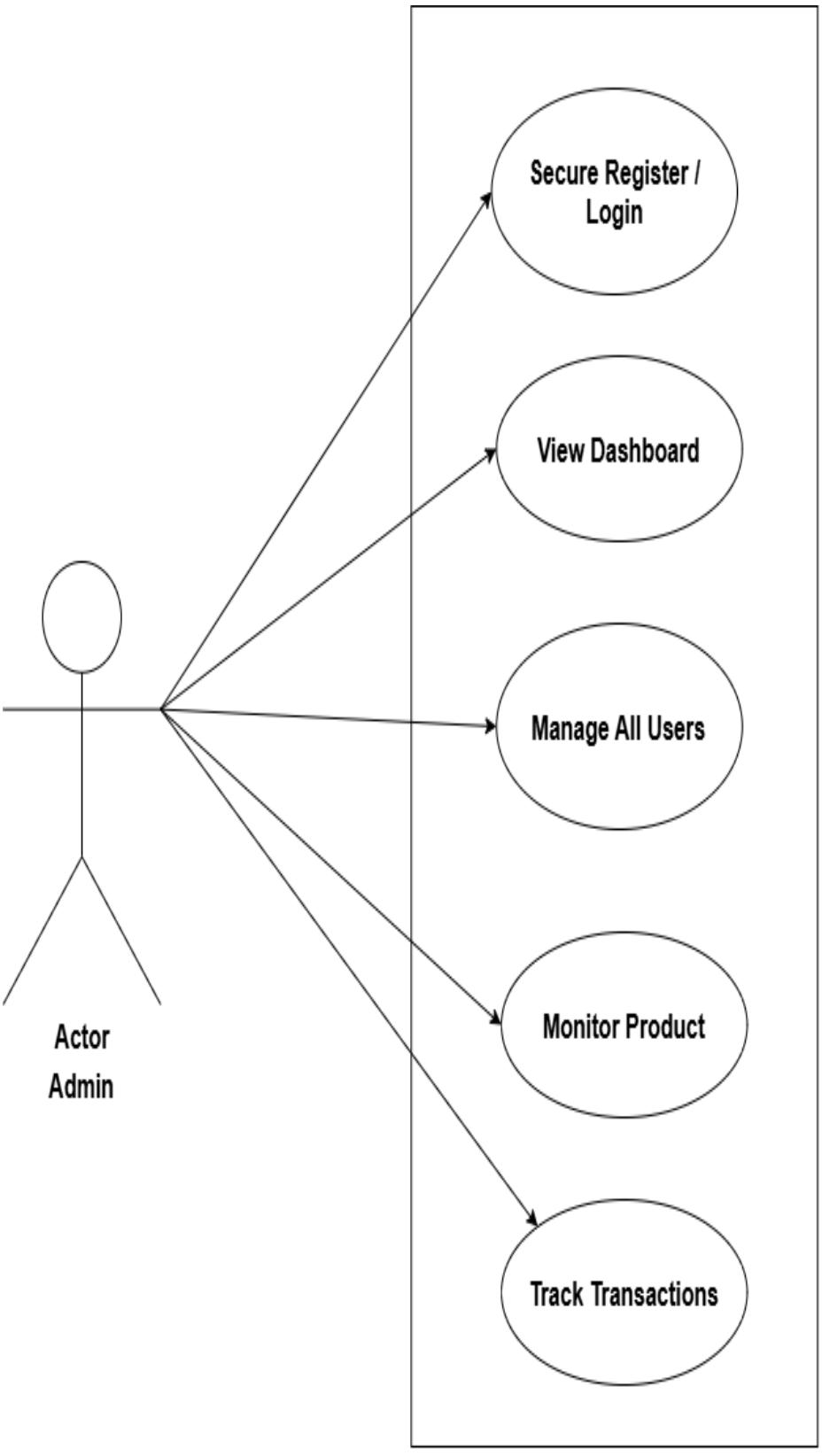
4.3 Component Diagram

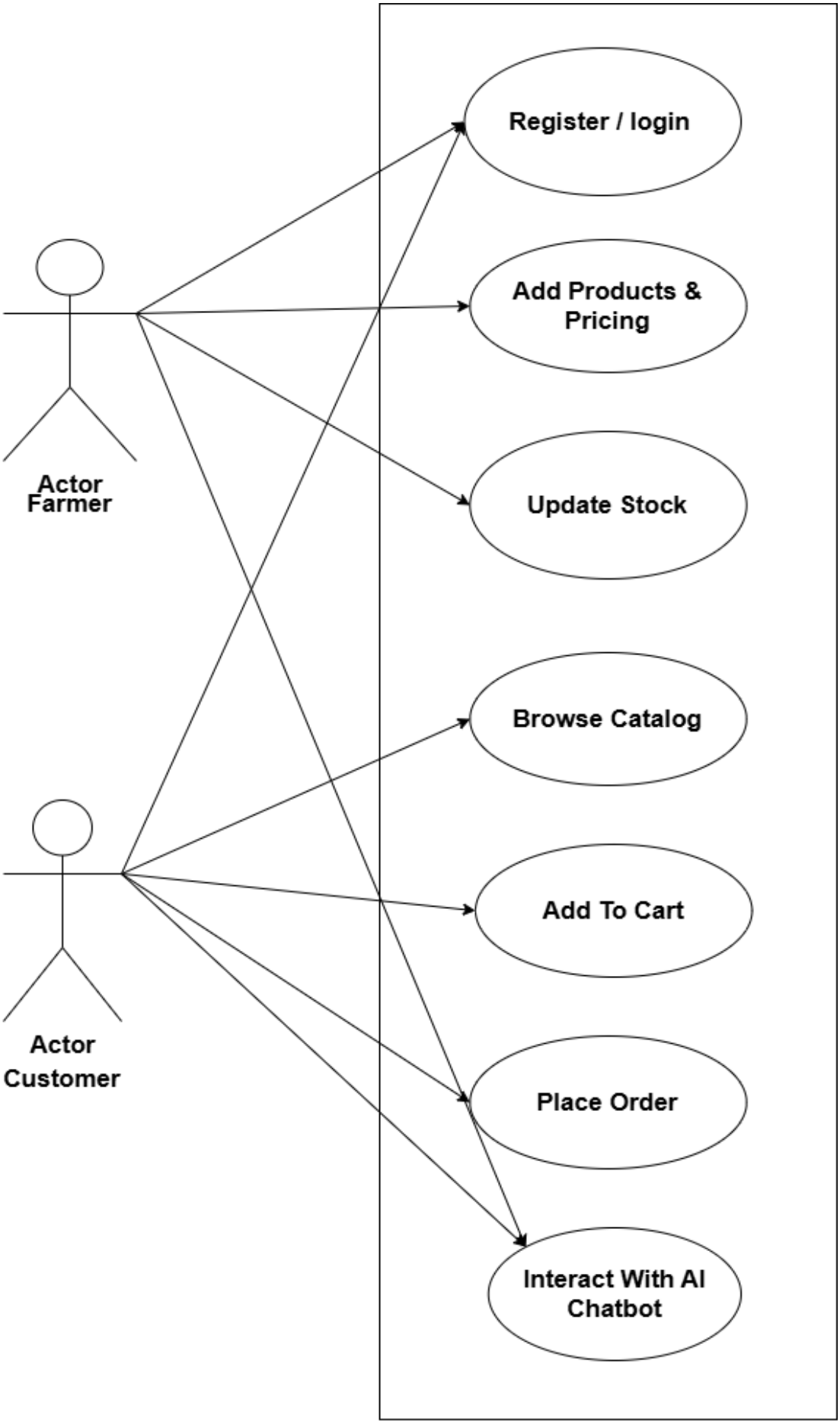


4.4 Deployment Diagram

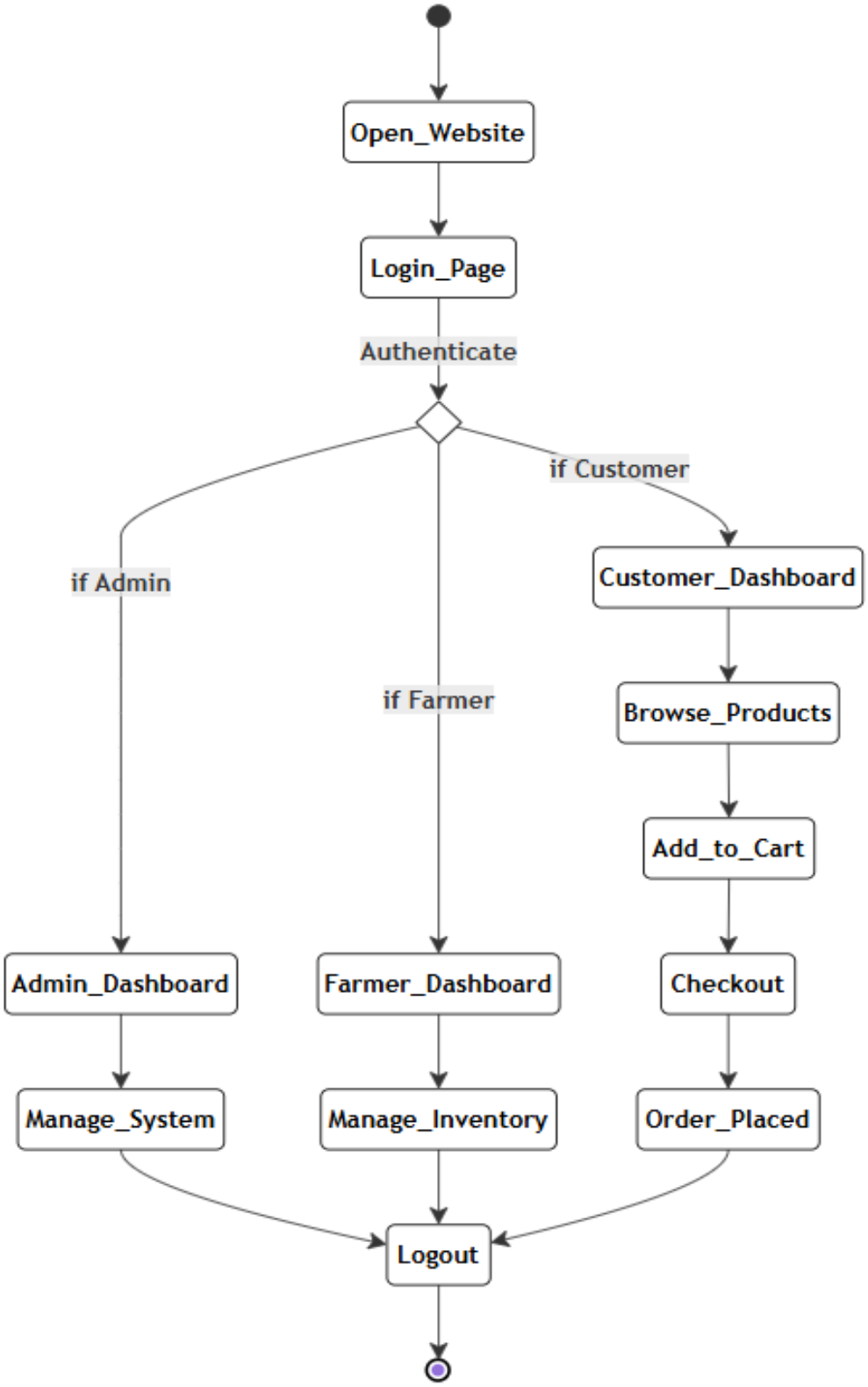


4.5 Use Case Diagram

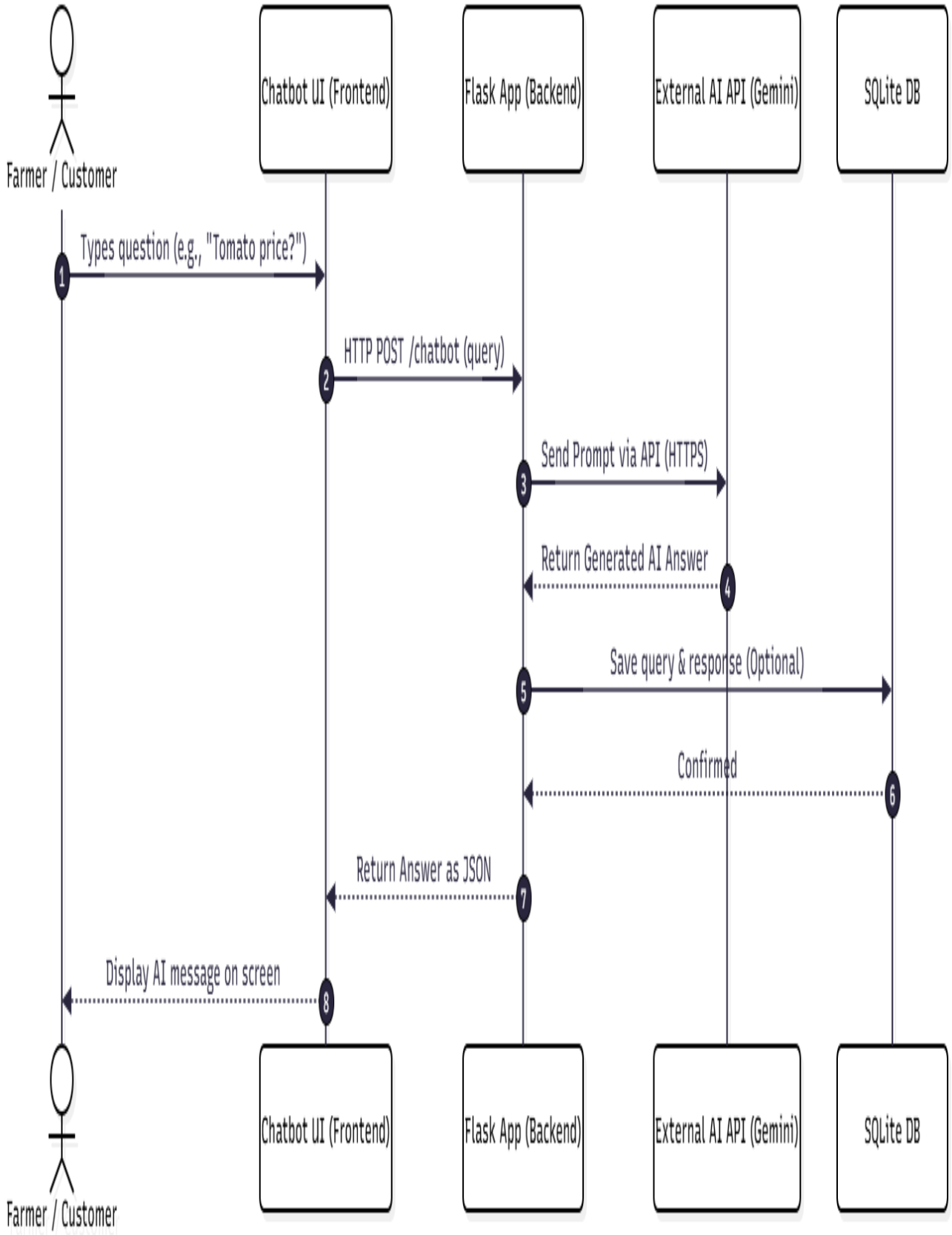



Register / login
Add Products & Pricing
Update Stock
Browse Catalog
Add To Cart
Place Order
Interact With AI Chatbot
Actor Farmer
Actor Customer

4.7 Activity Diagram



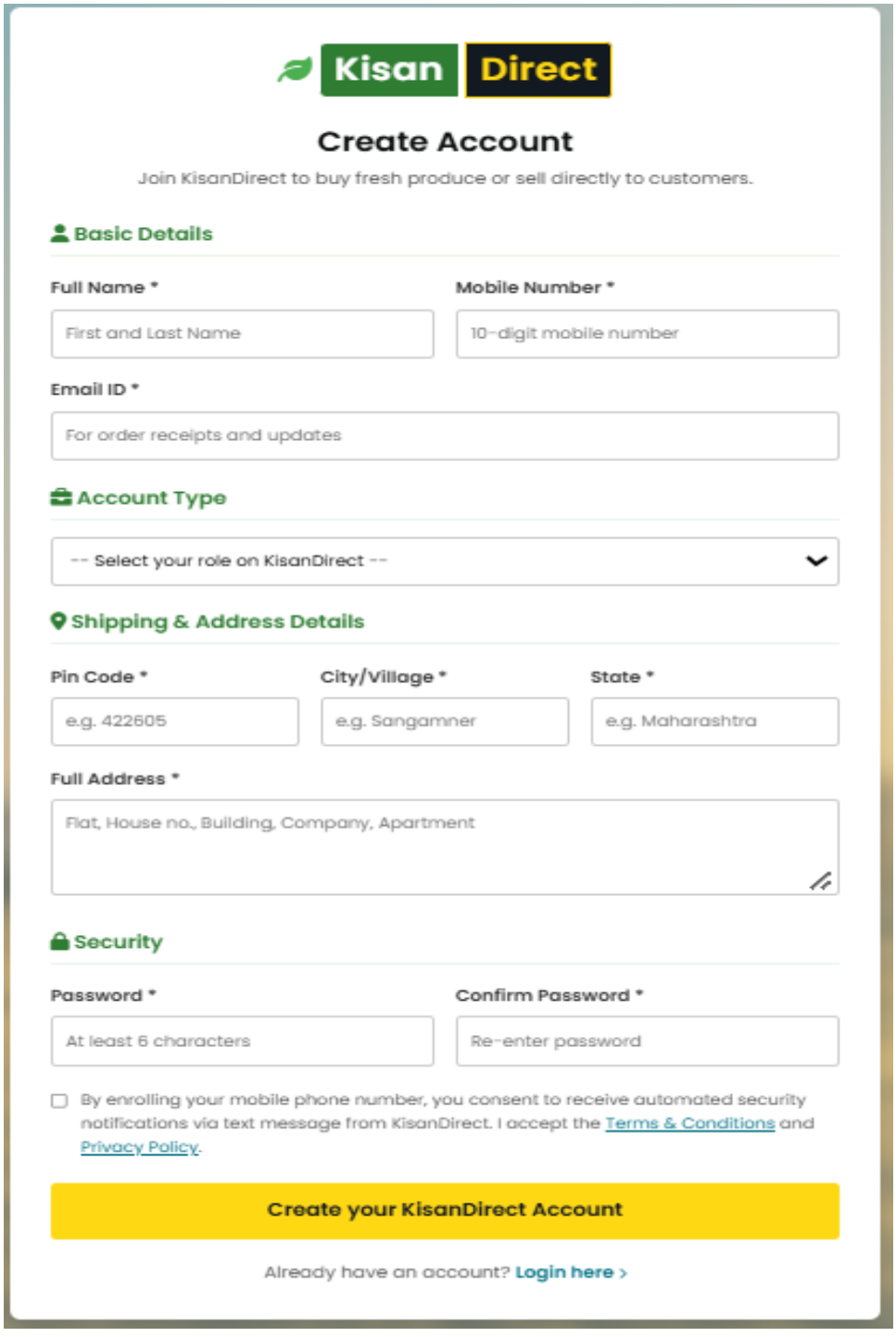
4.8 Sequence Diagram



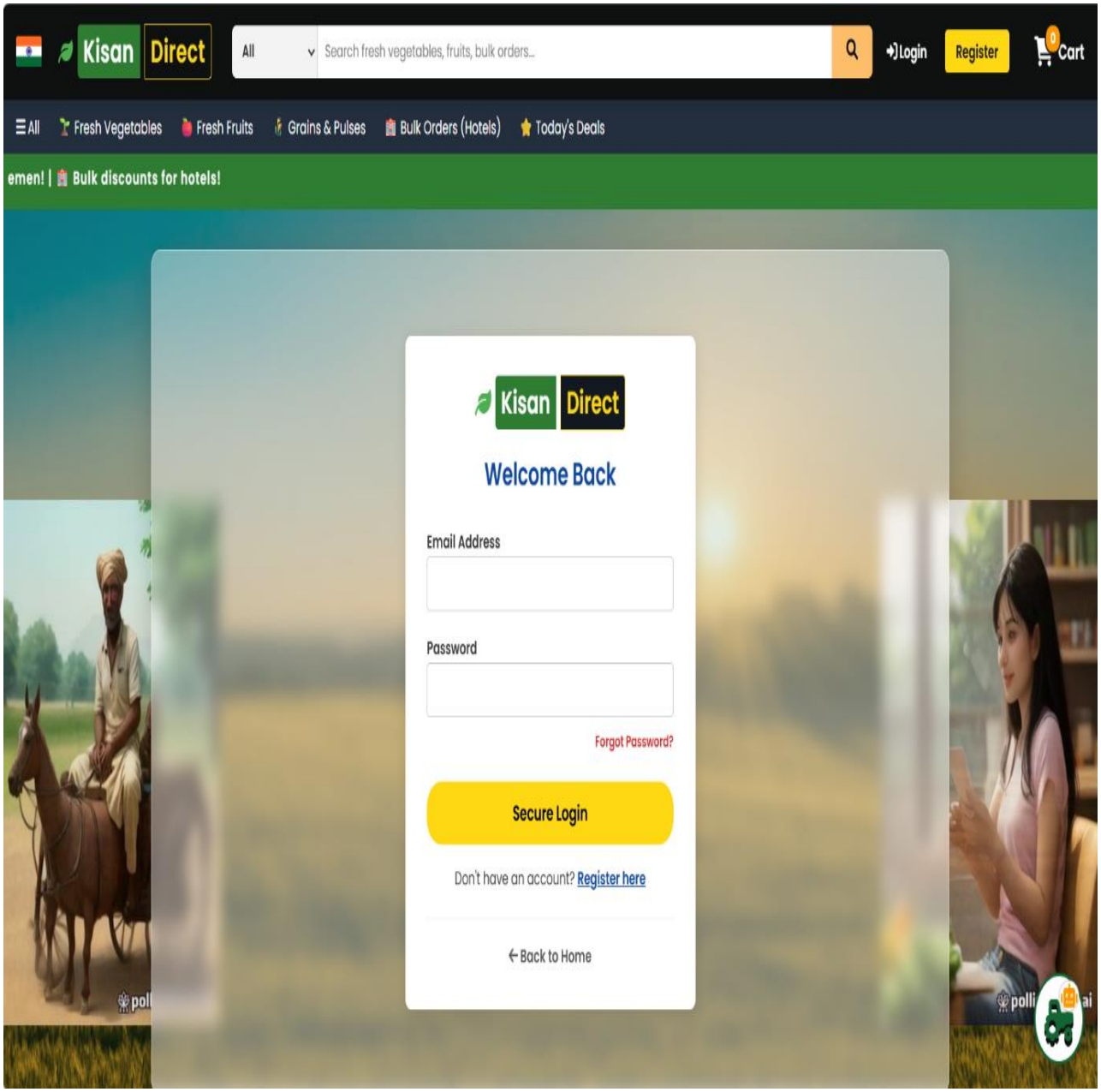
5.Input and Output screen Layout

❖ **Input Screens:-**

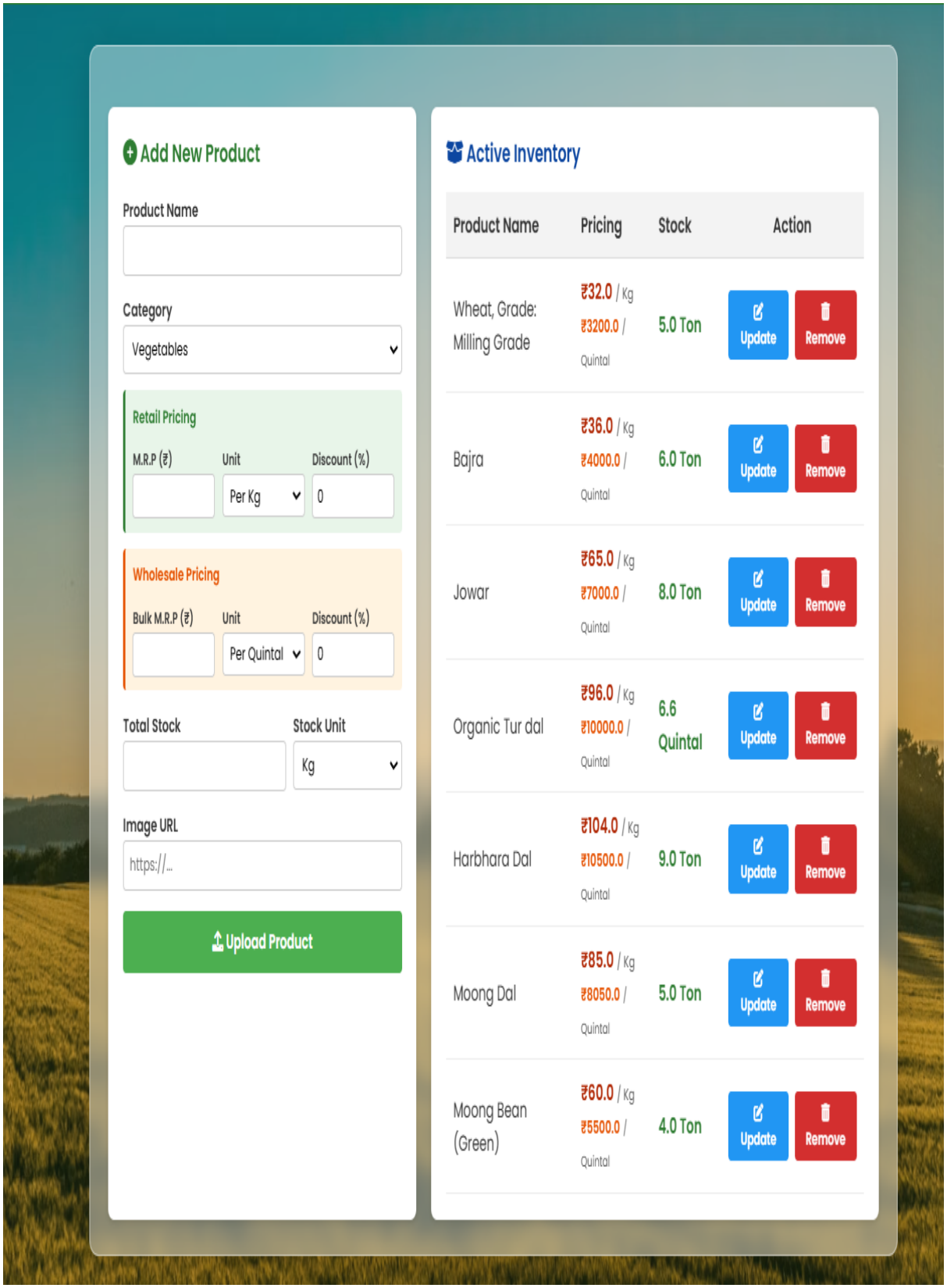
1.User Registration (Sign Up) Page:



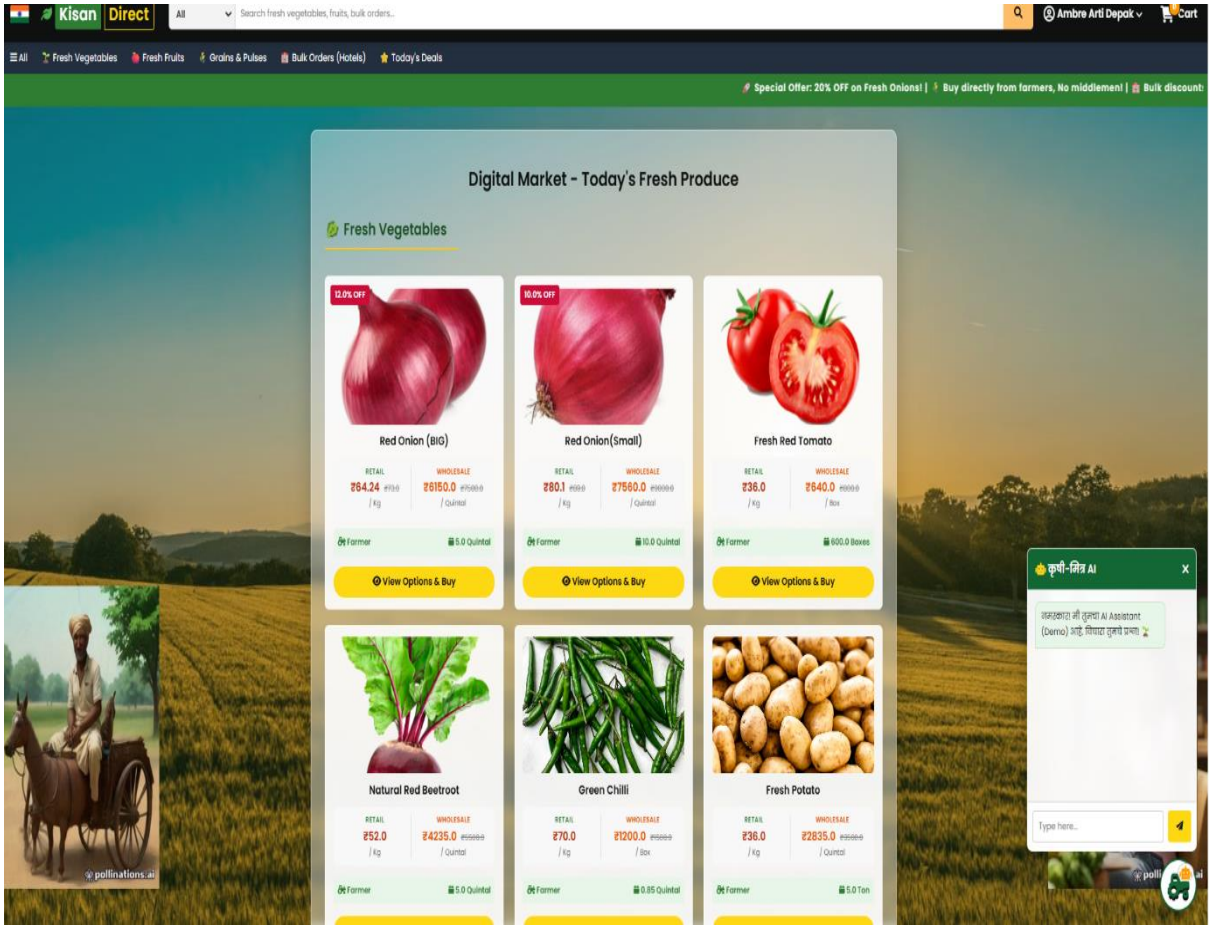
2.User Login Page:

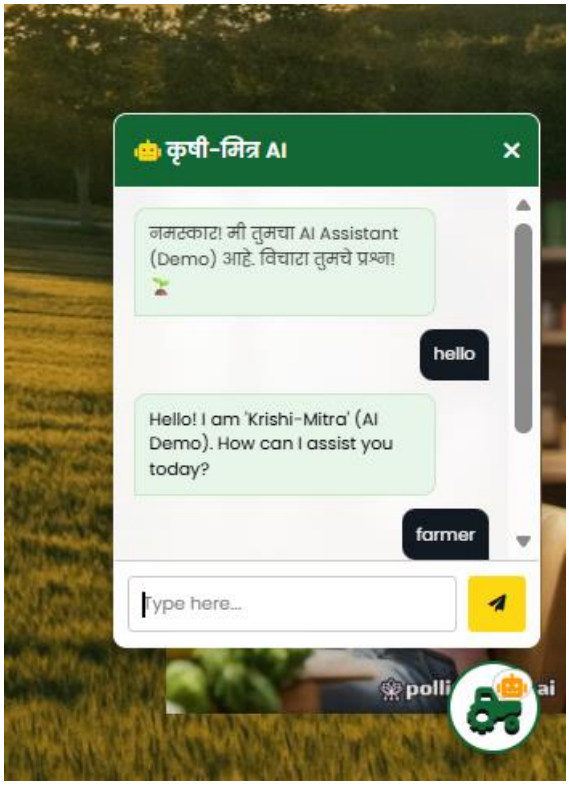


3.Farmer Dashboard - 'Add New Product' Form



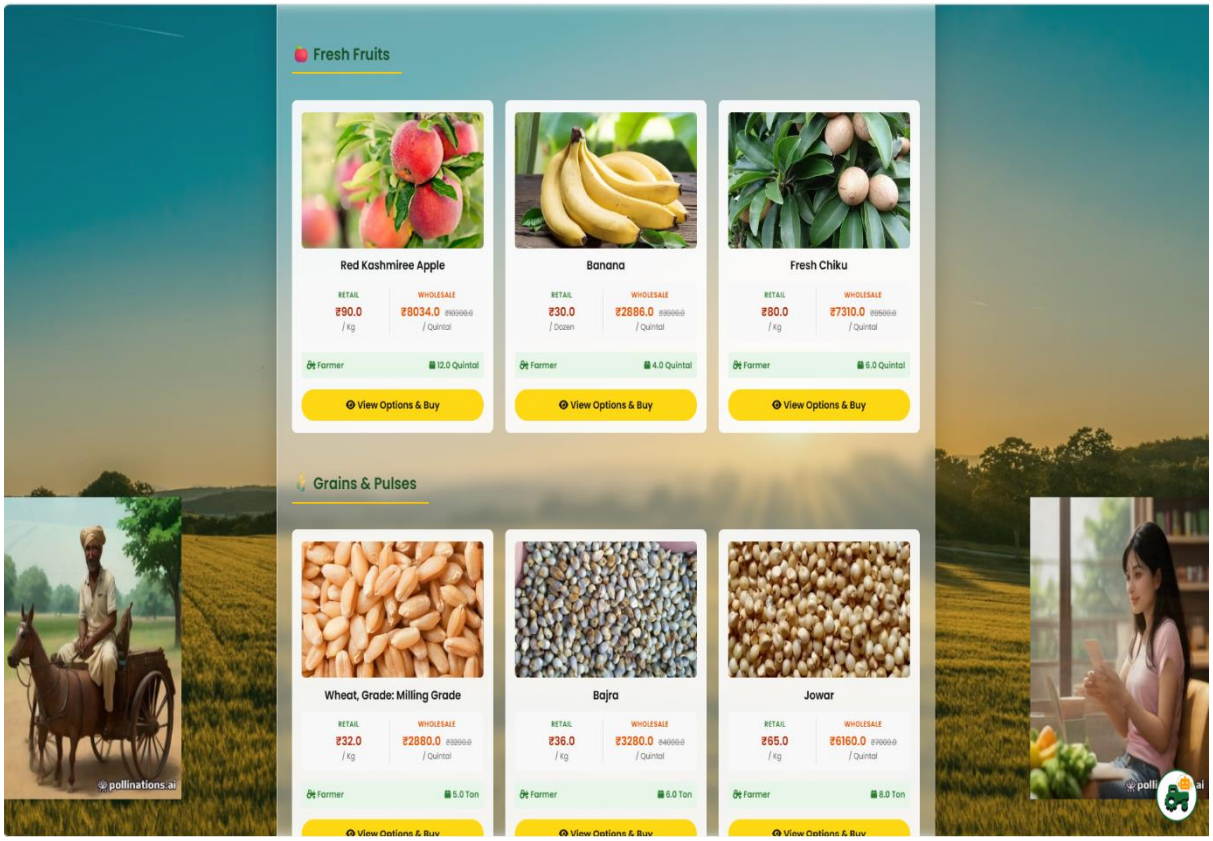
4.AI Chatbot Input Box (Krushi-Mitra AI):

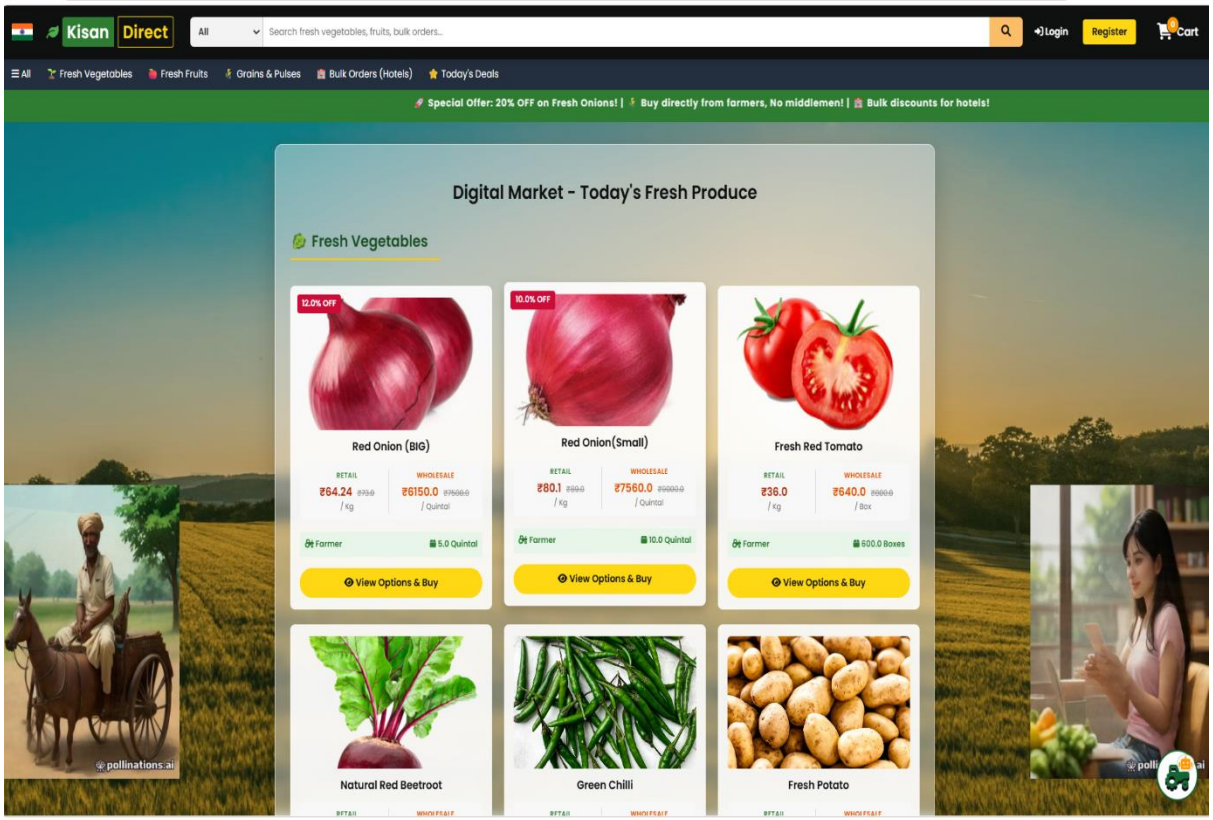




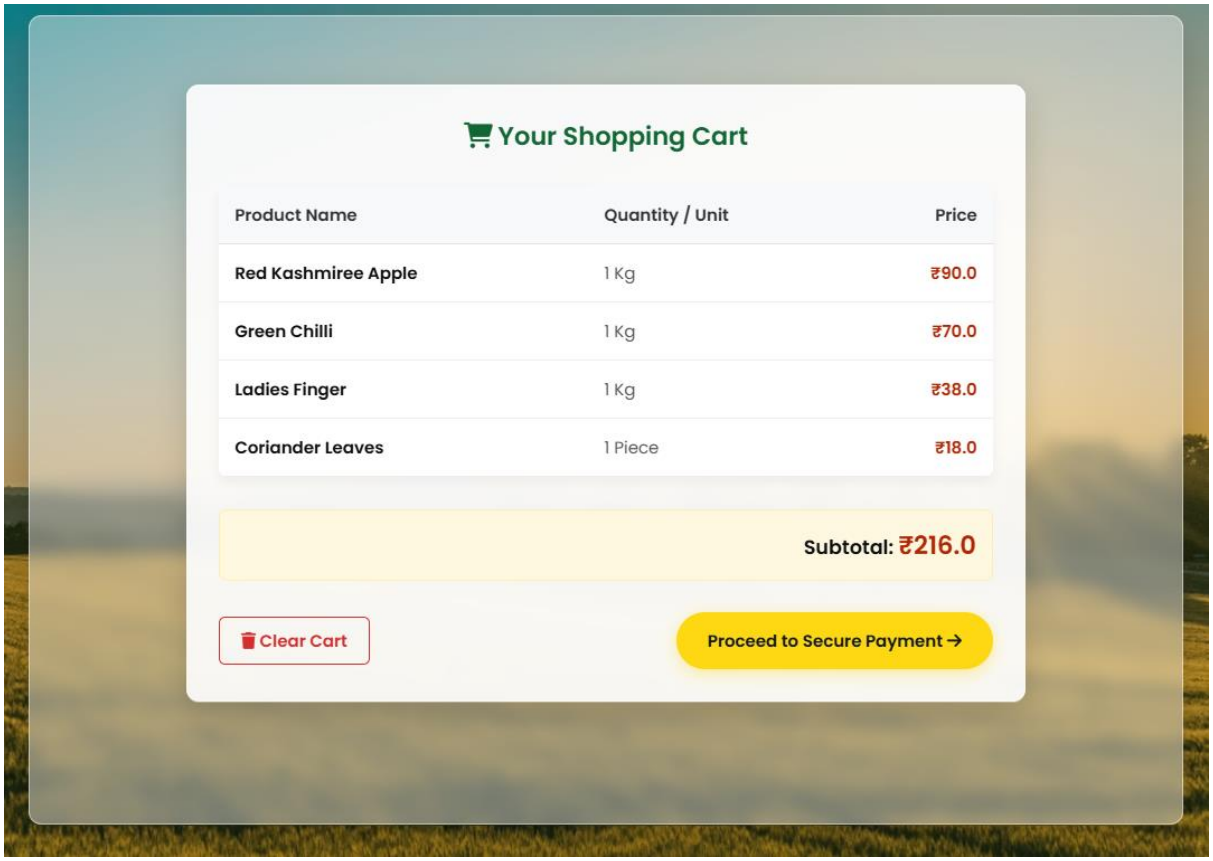
❖ **Output Screens:**

1.Home Page (Digital Market):

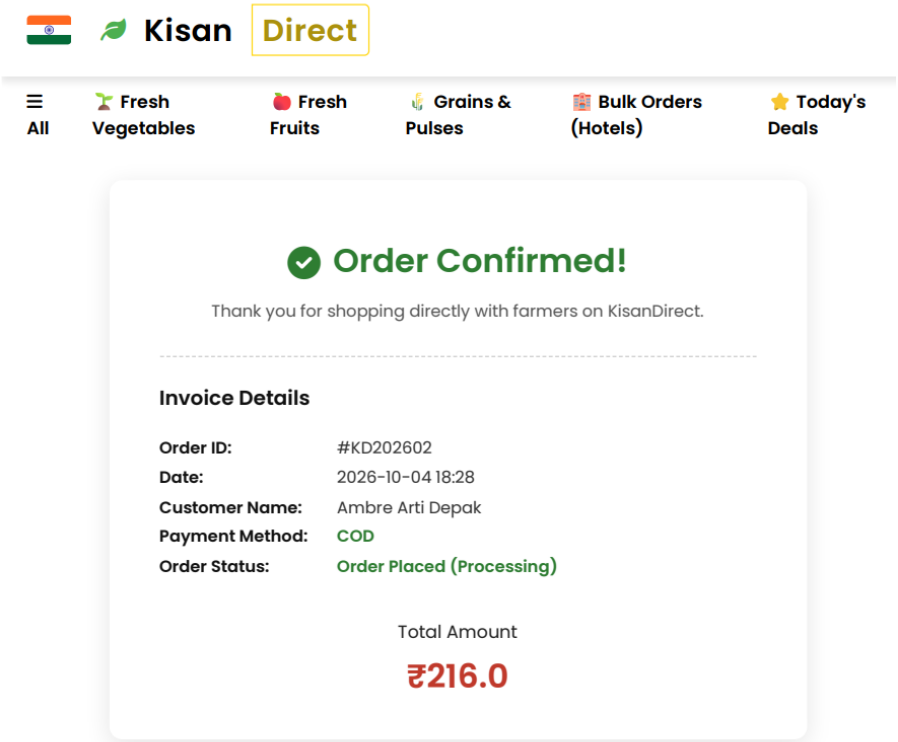




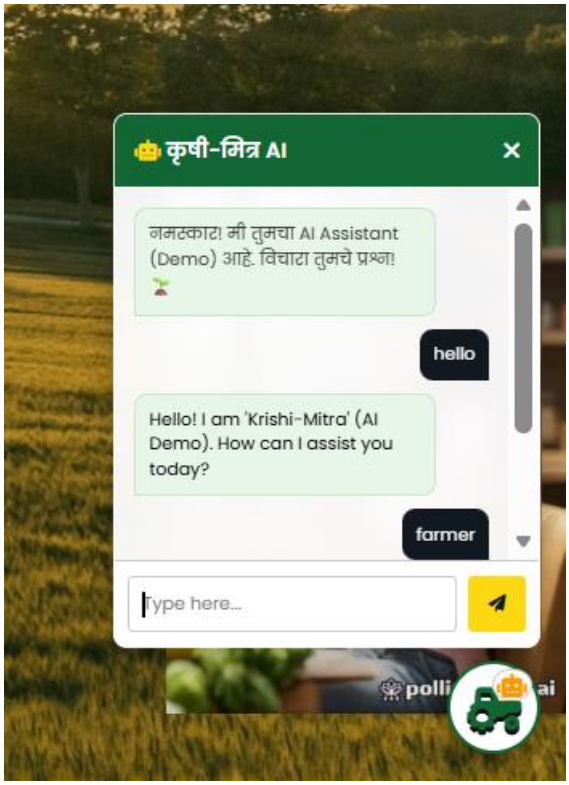
2.Shopping Cart Page:



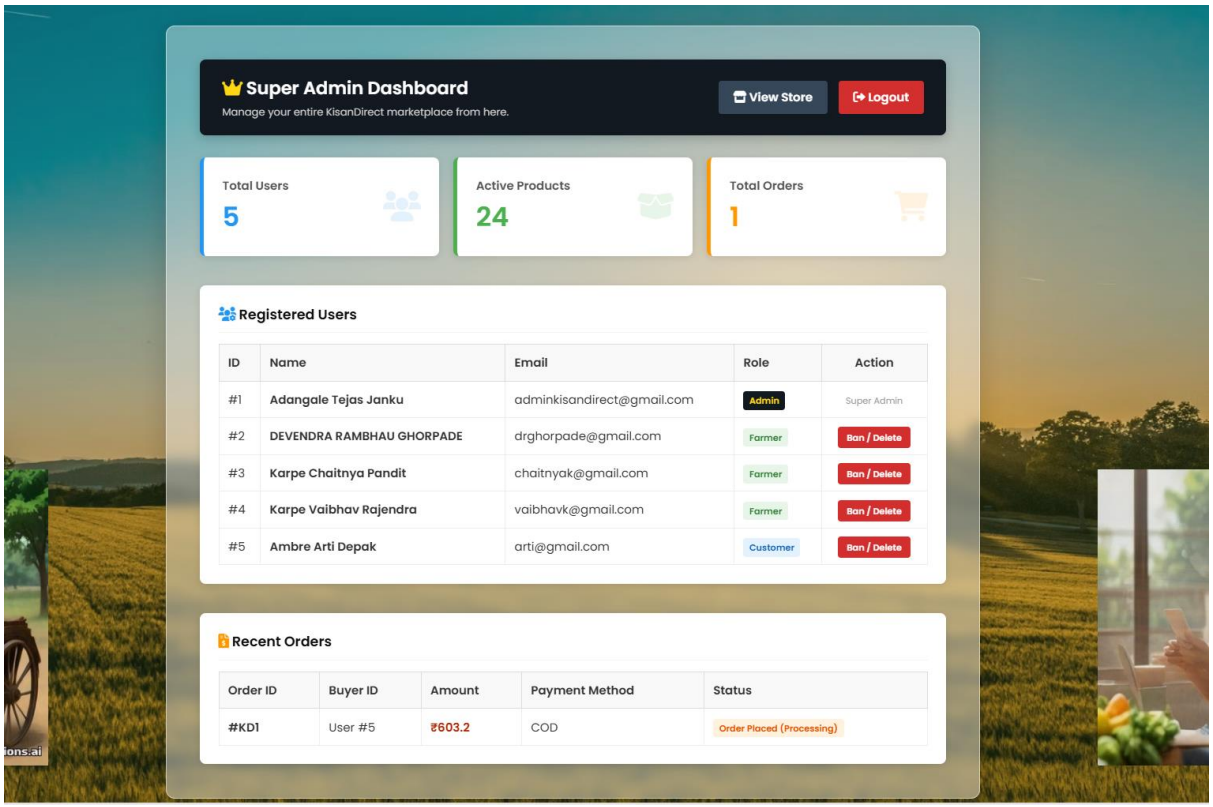
3.Order Invoice / Billing Screen:



4.AI Chatbot Response (Smart Output):



5.Admin Dashboard:



6. Bibliography / References

❖ **Overview:** During the research, design, and development of the "KisanDirect - Agri-Tech Marketplace" project, the following resources, official documentations, and web portals were referred to for technical guidance, UI/UX design, and the seamless integration of Artificial Intelligence.

6.1 Official Documentation & Technical Guides

- **Python Documentation:** Official Python 3.x documentation for backend logic and server-side scripting.
    - *URL:* https://docs.python.org/3/
- **Flask Framework:** Official documentation for setting up the WSGI web application, routing, and session management.
    - *URL:* https://flask.palletsprojects.com/
- **Flask-SQLAlchemy:** Reference guide for integrating the SQLite database and using Object Relational Mapper (ORM) in Flask.
    - *URL:* https://flask-sqlalchemy.palletsprojects.com/
- **SQLite Documentation:** Official reference for lightweight, serverless relational database management.
    - *URL:* https://www.sqlite.org/docs.html
- **Google Gemini API / OpenAI Documentation (AI Integration):** Official API guides referenced for generating API keys, prompt engineering, and integrating the "Krushi-Mitra" AI chatbot into the Flask backend.
    - *URL:* https://ai.google.dev/docs/ (or https://platform.openai.com/docs/)

6.2 Books and Academic References

- **Flask Web Development:** *Developing Web Applications with Python* by Miguel Grinberg. (Used for understanding advanced routing, application structure, and template rendering).
- **HTML and CSS:** *Design and Build Websites* by Jon Duckett. (Used as a reference for structuring clean, responsive frontend layouts).

6.3 Web Resources, Tutorials & Libraries

- **W3Schools:** Used for quick references regarding HTML5 structures, CSS3 styling (including Glassmorphism concepts), and basic JavaScript functionalities for chatbot API calls.
    - *URL:* https://www.w3schools.com/
- **MDN Web Docs (Mozilla Developer Network):** Extensively referred to for advanced CSS properties (like backdrop-filter, z-index, and Flexbox/Grid layouts) and asynchronous JavaScript (Fetch API).
    - *URL:* https://developer.mozilla.org/
- **Font Awesome:** Utilized for integrating scalable vector icons (e.g., shopping cart, user profile, tractor, AI robot/chat icons) into the user interface.
    - *URL:* https://fontawesome.com/
- **Stack Overflow:** Acted as a primary community forum for debugging coding errors, resolving CSS layout overlaps, and optimizing backend logic.
    - *URL:* https://stackoverflow.com/

6.4 Tools & Platforms

- **Visual Studio Code (VS Code):** Documentation and extension guides for the primary Integrated Development Environment (IDE) used for the project build.
- **Draw.io / Mermaid.js:** Sourced for creating, writing, and rendering the advanced UML diagrams (ER, Class, Object, Component, Deployment, Use Case, Activity, and Sequence diagrams) presented in the System Design chapter.
    - *URL:* https://app.diagrams.net/
- **Unsplash:** Sourced high-quality, royalty-free agricultural background images to enhance the visual appeal of the platform.